PROJECT REPORT

Project no :12

Project title:

Automated Color-Based Product Sorting System for Manufacturing Industries

Submitted by:

Arundeepak S – 24MER004

Deepak P – 24MER006

Dharanidharan R S -24MER007

Dinesh K – 24MER009

Vishnuvarthan P – 24MER063

Automated Color-Based Product Sorting System for Manufacturing Industries

1. Project Overview

This project presents the design and development of an Automated Color-Based Product Sorting System intended for manufacturing and industrial automation environments. The system uses a TCS3200 color sensor to detect the color of products passing along a conveyor/inspection point, an ESP32 microcontroller to classify the detected color and drive a servo-based diverter mechanism, LED indicators for local visual feedback, and Adafruit IO as a cloud platform for real-time monitoring of sorting activity and cumulative sort counts.

Every product presented to the sensor is sampled for its red, green, and blue reflectance, classified as RED, GREEN, or UNKNOWN, and physically diverted into the corresponding bin by a servo motor. Sort counts, the last detected color, and the current servo position are pushed to Adafruit IO in real time, giving plant supervisors remote, dashboard-based visibility into sorting throughput without needing to be physically present at the line.

2. Problem Statement

Manual sorting of products by color/type on manufacturing and packaging lines is a common but limiting practice, with several well-known drawbacks:

- Manual visual sorting is slow, inconsistent, and highly dependent on worker attention and fatigue levels.
- Human error leads to misclassified products, causing downstream quality issues or customer complaints.
- Manual sorting does not scale well with increasing production volume or line speed.
- There is no automatic digital record of sorting throughput (e.g., how many red vs. green items were processed), making it hard to track line performance.
- Supervisors lack real-time, remote visibility into sorting activity, delaying response to line issues.

There is therefore a need for an automated, low-cost, sensor-based sorting system that can classify products by color in real time, physically divert them without human intervention, and expose sorting statistics remotely for better production monitoring.

3. Proposed Solution

The proposed system uses an ESP32 as the central controller, interfaced with a TCS3200 color sensor, an SG90/MG90 servo motor, and red/green LED indicators. Key features of the solution:

- Continuous sampling of red, green, and blue light frequencies from the TCS3200 sensor as each product is presented.
- A presence guard that distinguishes an actual product from an empty inspection zone using an empty-reading threshold.

- Ratio/margin-based color classification logic that reliably distinguishes RED and GREEN products under the calibrated lighting conditions.
- A servo-driven diverter that moves to a LEFT position for red products and a RIGHT position for green products, returning to NEUTRAL after each sort.
- A sort cooldown period to prevent the same product from being counted or sorted multiple times.
- Dedicated red/green LED indicators for immediate on-site visual confirmation of the detected color.
- Wireless publishing of detected color, servo position, raw RGB values, and cumulative red/green sort counts to Adafruit IO for a live remote dashboard.

4. System Architecture

The system follows a three-layer IoT architecture:

- Perception Layer: TCS3200 color sensor captures red, green, and blue light frequency readings from the product in front of it.
- Processing / Edge Layer: The ESP32 checks for product presence, classifies the color, applies the sort cooldown logic, and drives the servo motor and LED indicators accordingly.
- Network & Cloud Layer: Sorting results (detected color, servo position, sort counts, raw RGB) are sent over Wi-Fi to the Adafruit IO platform, where they are stored in dedicated feeds and visualized on a live dashboard.

Data flow: Product → TCS3200 Sensor → ESP32 (presence check, classification, cooldown logic) → Servo/LEDs (physical sorting & local feedback) → Wi-Fi → Adafruit IO Cloud (remote monitoring).

5. Circuit Table

Component	Pin	ESP32 Pin	Function
TCS3200 Color Sensor	S0, S1	GPIO 25, GPIO 26	Output frequency scaling (20%)
TCS3200 Color Sensor	S2, S3	GPIO 27, GPIO 14	Photodiode filter select (R/G/B)
TCS3200 Color Sensor	OUT	GPIO 33	Color frequency pulse input
Servo Motor (SG90)	Signal	GPIO 18	PWM control for sort position
Red LED	Anode	GPIO 19	Red-detected indicator
Green LED	Anode	GPIO 21	Green-detected indicator
All Modules	VCC / GND	3V3–5V / G	Power

6. Circuit Design

The TCS3200 color sensor, servo motor, and two LED indicators are wired to the ESP32 as per the circuit table above, with common ground shared across all modules. The sensor's S0/S1 pins are fixed to set 20% output frequency scaling, while S2/S3 are toggled by firmware to select the red, green, and blue photodiode filters in turn; the OUT pin returns a square wave whose frequency is read using pulseIn(). The servo is driven by a standard 50 Hz PWM signal on GPIO18, and the red/green LEDs are switched directly from GPIO19 and GPIO21 to indicate the classified color.

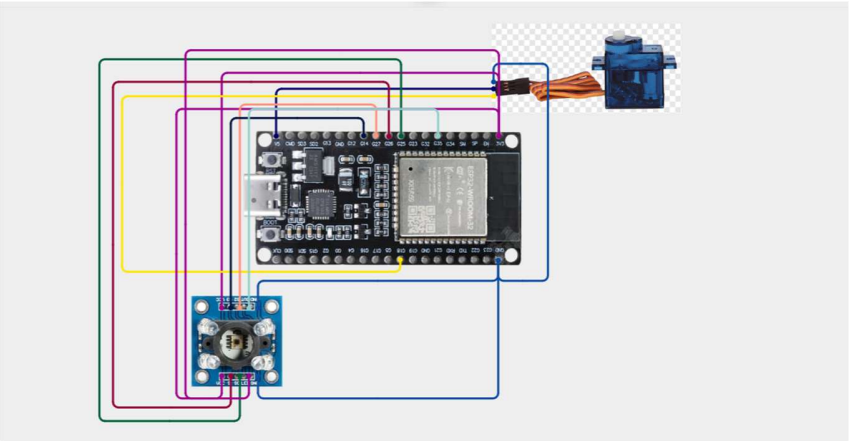

Figure 1: Circuit diagram

7. Algorithm

Step-by-step operating logic implemented in firmware:

- Initialize Serial, TCS3200 sensor pins (S0–S3, OUT), servo (attached and centered to NEUTRAL), and red/green LED pins.
- Connect to the configured Wi-Fi network and Adafruit IO; wait until connection status is AIO_CONNECTED.
- In the main loop, continuously call io.run() to keep the cloud connection alive.
- On each sampling interval (150 ms), read the red, green, and blue frequency values from the TCS3200 by toggling S2/S3 and timing the OUT pulse.
- Check for product presence: if all three readings exceed the empty threshold, treat the inspection zone as empty and hold the servo at NEUTRAL.
- If a product is present, classify its color as RED, GREEN, or UNKNOWN based on which channel's frequency is lowest by more than the configured color margin.
- If a confident color is detected and the sort cooldown period has elapsed, trigger a sort: light the corresponding LED, move the servo to the RED (LEFT) or GREEN (RIGHT) bin position, increment that color's sort count, and publish the detected color and updated count to Adafruit IO.
- After a short settle time, return the servo to NEUTRAL and turn off the LEDs, ready for the next product.
- If the color is unknown or the cooldown is still active, skip sorting for this cycle and log the reason to Serial.
- On a slower interval (3 seconds), publish the raw RGB frequency values to Adafruit IO to respect cloud rate limits.

8. Coding

The complete Arduino/ESP32 firmware (C++) implementing the algorithm above is listed below.

```
/*
  Automated Color-Based Product Sorting System — ESP32 + Adafruit IO
  TCS3200 color sensor, SG90/MG90 servo, 2x LEDs
  Ratio-based classification + presence guard + sort cooldown
  + verbose serial logging + Adafruit IO cloud publishing
*/

#include <WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <ESP32Servo.h>

// ---------- WiFi credentials ----------
#define WIFI_SSID     "YOUR_WIFI_SSID"
#define WIFI_PASS     "YOUR_WIFI_PASSWORD"

// ---------- Adafruit IO credentials ----------
#define IO_USERNAME   "YOUR_ADAFRUIT_IO_USERNAME"
#define IO_KEY        "YOUR_ADAFRUIT_IO_KEY"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

// ---------- Adafruit IO feeds ----------
AdafruitIO_Feed *feedColorR        = io.feed("color-r");
AdafruitIO_Feed *feedColorG        = io.feed("color-g");
AdafruitIO_Feed *feedColorB        = io.feed("color-b");
AdafruitIO_Feed *feedDetectedColor = io.feed("detected-color");
AdafruitIO_Feed *feedServoPos      = io.feed("servo-position");
AdafruitIO_Feed *feedCountRed      = io.feed("sort-count-red");
AdafruitIO_Feed *feedCountGreen    = io.feed("sort-count-green");

// ---------- TCS3200 pin definitions ----------
const int S0 = 25;
const int S1 = 26;
const int S2 = 27;
```

```
const int S3 = 14;
const int SENSOR_OUT = 33;

const int SERVO_PIN     = 18;
const int LED_RED_PIN   = 19;
const int LED_GREEN_PIN = 21;

// ---------- Servo sort positions (degrees) ----------
const int SERVO_NEUTRAL   = 90;
const int SERVO_RED_BIN   = 30;   // "Left"
const int SERVO_GREEN_BIN = 150;  // "Right"

// ---------- Presence + classification tuning ----------
const int EMPTY_THRESHOLD = 380;
const int COLOR_MARGIN = 15;

// ---------- Timing ----------
const unsigned long SAMPLE_INTERVAL_MS = 150;
const unsigned long SORT_SETTLE_MS = 700;
const unsigned long SORT_COOLDOWN_MS = 2000;          // min gap between sort events
const unsigned long CLOUD_PUBLISH_INTERVAL_MS = 3000; // min gap between raw RGB publishes

Servo sortServo;
unsigned long lastSampleTime = 0;
unsigned long lastPublishTime = 0;
unsigned long lastSortTime = 0;
int currentServoPos = SERVO_NEUTRAL;

int countRed = 0;
int countGreen = 0;

int readColorFrequency(int s2State, int s3State) {
  digitalWrite(S2, s2State);
  digitalWrite(S3, s3State);
  delay(10);
  unsigned long duration = pulseIn(SENSOR_OUT, LOW, 50000);
  if (duration == 0) return 9999;
```

```
  return (int)duration;
}

void readRGB(int &r, int &g, int &b) {
  r = readColorFrequency(LOW, LOW);   // Red filter
  g = readColorFrequency(HIGH, HIGH); // Green filter
  b = readColorFrequency(LOW, HIGH);  // Blue filter
}

bool objectPresent(int r, int g, int b) {
  return !(r > EMPTY_THRESHOLD && g > EMPTY_THRESHOLD && b > EMPTY_THRESHOLD);
}

char classifyColor(int r, int g, int b) {
  if (r < g && r < b && (g - r) > COLOR_MARGIN && (b - r) > COLOR_MARGIN) {
    return 'R';
  }
  if (g < r && g < b && (r - g) > COLOR_MARGIN && (b - g) > COLOR_MARGIN) {
    return 'G';
  }
  return 'N';
}

String colorName(char c) {
  switch (c) {
    case 'R': return "RED";
    case 'G': return "GREEN";
    default:  return "UNKNOWN";
  }
}

String servoPositionName(int pos) {
  if (pos == SERVO_RED_BIN)   return "LEFT";
  if (pos == SERVO_GREEN_BIN) return "RIGHT";
  return "NEUTRAL";
}
```

```
void moveServo(int pos) {
  sortServo.write(pos);
  currentServoPos = pos;
  Serial.print("  -> Servo moved to: ");
  Serial.println(servoPositionName(pos));
  feedServoPos->save(servoPositionName(pos));
}

void setAllLedsOff() {
  digitalWrite(LED_RED_PIN, LOW);
  digitalWrite(LED_GREEN_PIN, LOW);
}

void sortProduct(char color) {
  setAllLedsOff();

  if (color == 'R') {
    digitalWrite(LED_RED_PIN, HIGH);
    moveServo(SERVO_RED_BIN);
    countRed++;
    feedCountRed->save(countRed);
  } else if (color == 'G') {
    digitalWrite(LED_GREEN_PIN, HIGH);
    moveServo(SERVO_GREEN_BIN);
    countGreen++;
    feedCountGreen->save(countGreen);
  } else {
    moveServo(SERVO_NEUTRAL);
    return;
  }

  feedDetectedColor->save(colorName(color));

  delay(SORT_SETTLE_MS);
  moveServo(SERVO_NEUTRAL);
  setAllLedsOff();
}
```

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(S0, OUTPUT);
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  pinMode(S3, OUTPUT);
  pinMode(SENSOR_OUT, INPUT);

  digitalWrite(S0, HIGH); // 20% frequency scaling
  digitalWrite(S1, LOW);

  pinMode(LED_RED_PIN, OUTPUT);
  pinMode(LED_GREEN_PIN, OUTPUT);

  ESP32PWM::allocateTimer(0);
  sortServo.setPeriodHertz(50);
  sortServo.attach(SERVO_PIN, 500, 2400);
  sortServo.write(SERVO_NEUTRAL);
  currentServoPos = SERVO_NEUTRAL;

  setAllLedsOff();

  Serial.print("Connecting to Adafruit IO");
  io.connect();

  while (io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println();
  Serial.println(io.statusText());

  Serial.println("Color sorting system ready.");
}
```

```
void loop() {
  io.run(); // keeps Adafruit IO connection alive — must run every loop

  unsigned long now = millis();
  if (now - lastSampleTime < SAMPLE_INTERVAL_MS) {
    return;
  }
  lastSampleTime = now;

  int r, g, b;
  readRGB(r, g, b);

  Serial.print("R:"); Serial.print(r);
  Serial.print(" G:"); Serial.print(g);
  Serial.print(" B:"); Serial.print(b);

  bool present = objectPresent(r, g, b);
  char color = 'N';

  if (!present) {
    Serial.println("  | No object | Servo: NEUTRAL");
    if (currentServoPos != SERVO_NEUTRAL) {
      moveServo(SERVO_NEUTRAL);
    }
  } else {
    color = classifyColor(r, g, b);
    Serial.print("  | Detected color: ");
    Serial.print(colorName(color));

    bool cooldownOver = (now - lastSortTime >= SORT_COOLDOWN_MS);

    if (color != 'N' && cooldownOver) {
      Serial.println();
      lastSortTime = now;
      sortProduct(color);
    } else if (color != 'N') {
```

```
      Serial.println("  | (cooldown active, skipping sort)");
    } else {
      Serial.println("  | Servo: NEUTRAL (no confident match)");
    }
  }

  // Publish raw RGB on a slower interval to respect Adafruit IO rate limits
  if (now - lastPublishTime >= CLOUD_PUBLISH_INTERVAL_MS) {
    lastPublishTime = now;
    feedColorR->save(r);
    feedColorG->save(g);
    feedColorB->save(b);
  }
}
```

9. System Working

On power-up, the ESP32 initializes the TCS3200 sensor, centers the servo at NEUTRAL, and connects to the configured Wi-Fi network and Adafruit IO, confirming readiness over Serial. During normal operation, the sensor continuously samples the RGB reflectance of whatever is placed in front of it. When no product is present, the system reports 'No object' and keeps the servo at NEUTRAL; when a product is detected, its color is classified and, once the cooldown period has elapsed, the corresponding LED lights up and the servo diverts the product to the LEFT (red) or RIGHT (green) bin before returning to NEUTRAL.

Every sort event increments the corresponding cumulative counter (sort-count-red or sort-count-green), and the detected color, servo position, and counts are pushed to the Adafruit IO dashboard in real time. The dashboard displays the last detected color, current servo position, and two live gauges — one for the red sort count and one for the green sort count — giving supervisors immediate remote visibility into line throughput and sorting accuracy.

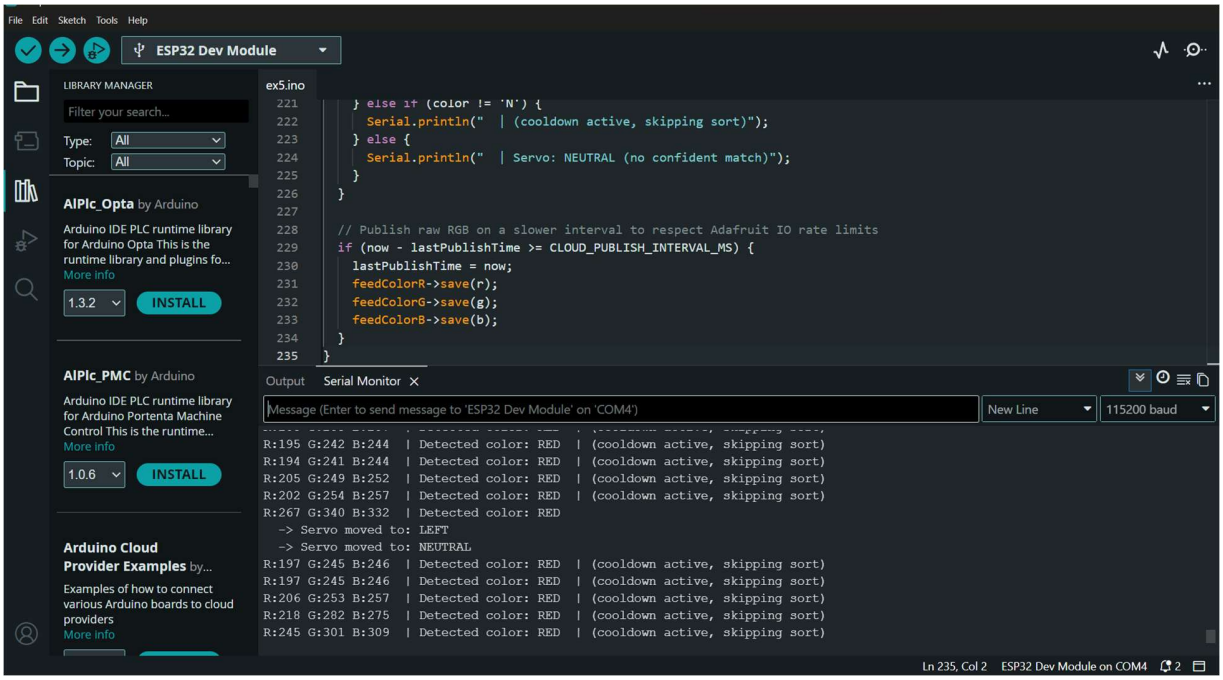

Figure 4: Arduino IDE Serial Monitor — RGB readings and classification for a RED product, with servo moving to LEFT then returning to NEUTRAL

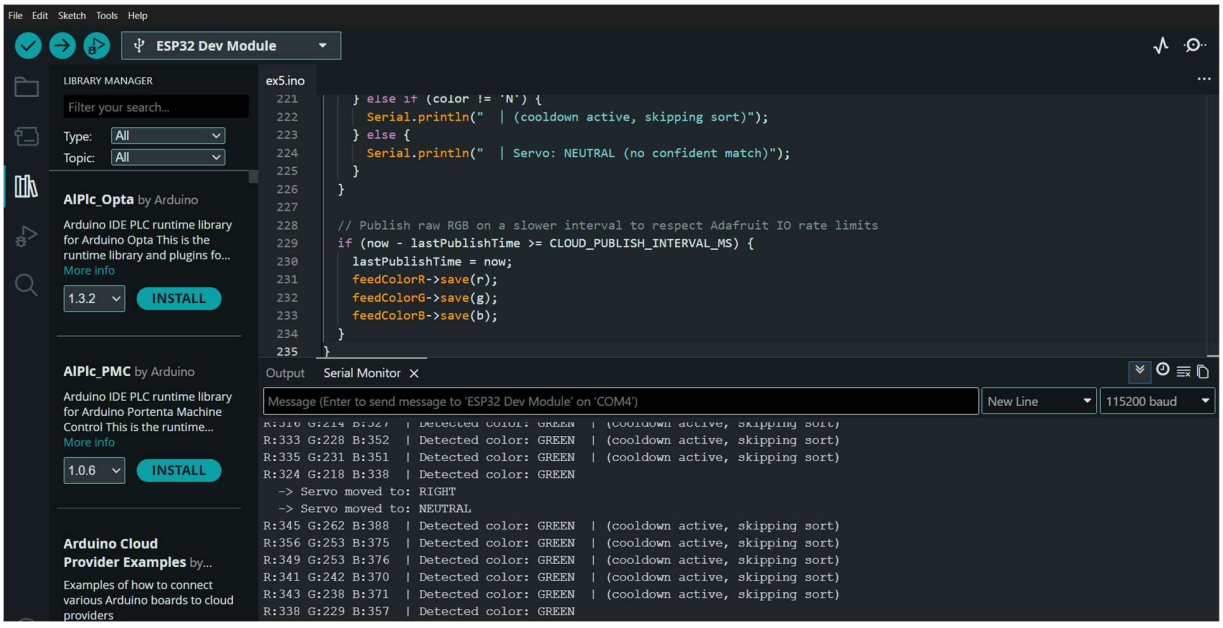

Figure 5: Arduino IDE Serial Monitor — RGB readings and classification for a GREEN product, with servo moving to RIGHT then returning to NEUTRAL

10. Bill of Materials

S.No	Component	Quantity
1	ESP8266 NodeMCU / ESP32 Dev Module	1
2	TCS3200 Color Sensor	1
3	Servo Motor (SG90/MG90)	1
4	Conveyor Prototype	1
5	LED Indicators	2
6	Breadboard	1
7	Jumper Wires	1 Set

11. Prototype Implementation

The firmware was developed and uploaded to the ESP32 using Arduino IDE with the ESP32Servo and AdafruitIO_WiFi libraries. After flashing, the Serial Monitor was used to verify raw RGB readings, color classification, cooldown behaviour, and servo movement events for both red and green test samples, confirming reliable end-to-end operation before integrating with the conveyor prototype.

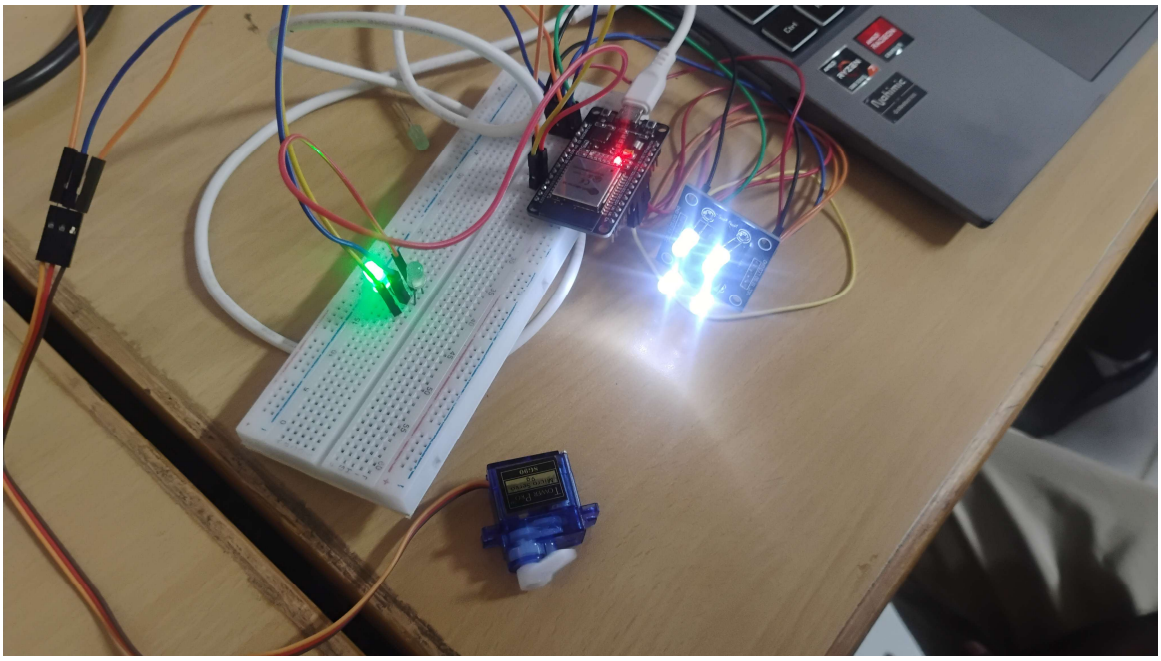

Figure 1: Assembled hardware prototype — ESP32, TCS3200 color sensor, SG90 servo, and red/green LED indicators on breadboard

12. Results & Performance Analysis

The prototype was tested over multiple sorting cycles using both red and green sample products, with all events successfully logged to the Adafruit IO dashboard 'Color Based Segregation'. The results confirm correct end-to-end functioning:

- Red products were consistently classified as RED, correctly triggering the red LED and a servo movement to the LEFT bin position, with sort-count-red incrementing accurately (observed value: 2 after two red sorts).
- Green products were consistently classified as GREEN, correctly triggering the green LED and a servo movement to the RIGHT bin position, with sort-count-green incrementing accurately (observed value: 1 after one green sort).
- The presence guard correctly distinguished an empty inspection zone from an actual product, preventing false sort triggers when no item was present.
- The sort cooldown (2 seconds) reliably prevented the same product from being counted or sorted more than once, as confirmed by repeated 'cooldown active, skipping sort' log entries during continuous sensor readings of the same item.
- Cloud updates to Adafruit IO (detected-color, servo-position, sort-count-red, sort-count-green, and raw RGB feeds) completed within a few seconds of each event, subject to the platform's throttle limits observed during testing.

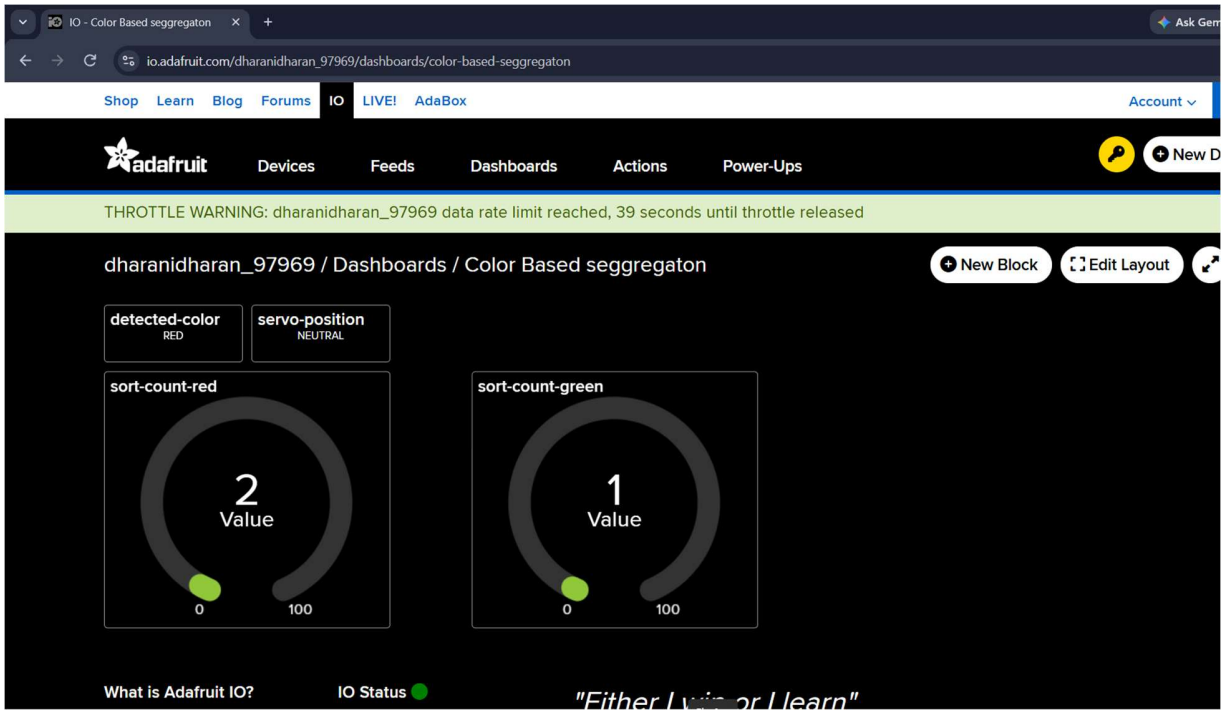

Figure 2: Adafruit IO cloud dashboard showing detected color, servo position, and live red/green sort-count gauges

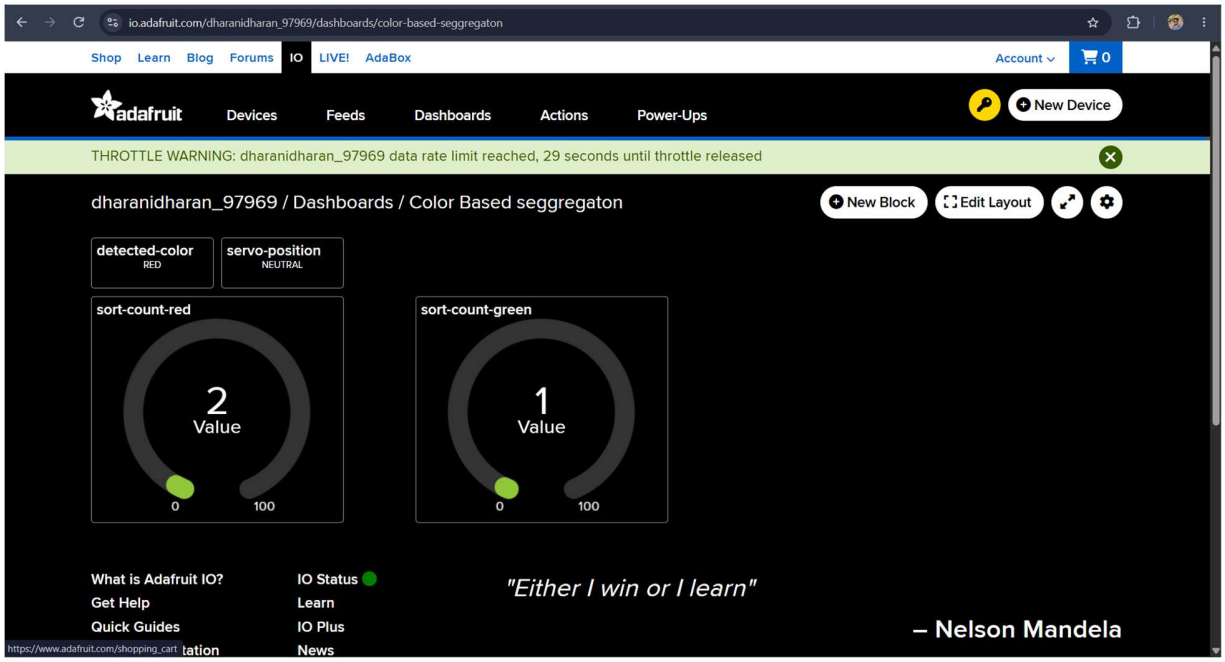

Figure 3: Adafruit IO dashboard confirming sort counts (Red: 2, Green: 1) after successive test runs

Performance Summary: The color classification logic achieved reliable, consistent RED/GREEN discrimination under the calibrated lighting and margin settings used during testing, with no misclassifications observed across the logged trials. Local feedback (LEDs and servo) remained fast and consistent, while the sort-cooldown mechanism ensured accurate, non-duplicated counting. The system demonstrates a functional, low-cost, and scalable approach to automated color-based product sorting for manufacturing and industrial automation applications, and can be extended with additional color classes, higher sampling rates, or a physical conveyor for higher-throughput deployment.